GitHub OIDC

Why kubeconfig, SA tokens & short-lived tokens all fail

The Bootstrap Problem — and why OIDC is the only real solution

What is kubeconfig?

A **kubeconfig file** is simply a file that allows you to connect to a Kubernetes cluster.

When you run a command like:

kubectl get pods

kubectl doesn't know anything by default. It needs:

- which cluster to connect to
- who you are
- how to authenticate

All of that information comes from the **kubeconfig file**.

So in simple terms:
kubeconfig is your access file to Kubernetes.

How is kubeconfig created?

Most of the time, you don't create it manually.

It gets generated automatically when you connect to a cluster.

For example, in your case with AWS EKS:

```
aws eks update-kubeconfig --name my-cluster
```

When you run this:

- AWS gives your cluster endpoint
- sets up authentication using IAM
- stores everything in your kubeconfig file

After this, your system is ready to run kubectl commands.

What does kubeconfig contain?

The file mainly has three things.

First, it has **cluster details** — basically where your Kubernetes cluster is running, including the API server endpoint.

Second, it has **user credentials** — this defines who you are and how you authenticate (like IAM role, token, or certificates).

Third, it has something called a **context** — which is just a mapping of: cluster + user + namespace.

This tells Kubernetes:
“Use this user to talk to this cluster in this namespace.”

What can you do with kubeconfig?

Once you have a valid kubeconfig file, you can fully interact with the cluster.

You can:

- deploy applications
- create services
- scale workloads

- delete resources

For example:

kubectl apply -f deployment.yaml

This will deploy your app to the cluster using the access defined in kubeconfig.

So yes — **kubeconfig gives you full access to the cluster.**

Important note (real-world)

Even though kubeconfig can be used to access the cluster and perform deployments, **sharing it directly is not a good practice**.

If someone gets your kubeconfig file, they can control your cluster depending on the permissions.

In production setups, instead of sharing kubeconfig:

- we use IAM roles
- OIDC (like in your GitHub Actions)
- short-lived credentials

The One Question That Explains Everything

Every authentication method for GitHub Actions → EKS comes down to one question:

The Bootstrap Question

How does GitHub Actions prove to AWS / Kubernetes that it is allowed to deploy?

Scenario 1 (kubeconfig): Store a credential → use it directly
Scenario 2 (SA long token): Store a credential → use it directly
Scenario 3 (SA short token): Store AWS keys → use them to generate a short K8s token
GitHub OIDC: Prove identity cryptographically → no stored credential needed

The first three all answer: 'trust me, I have the password'.
OIDC answers: 'verify my identity from a trusted source'.

Scenario 1 **kubeconfig Secret**

Export ~/.kube/config → base64 encode → store as KUBE_CONFIG_B64 in GitHub

How It Works

You export your local kubeconfig — which contains either your personal AWS credentials embedded, or a long-lived Kubernetes token. You base64 encode it and store it as a GitHub Secret. The pipeline decodes it at runtime.

```
# Export and encode
cat ~/.kube/config | base64 -w 0
# Store as KUBE_CONFIG_B64 in GitHub Secrets

# In the pipeline
echo "$KUBE_CONFIG_B64" | base64 -d > ~/.kube/config
kubectl apply -f k8s/ -n project
```

What Goes Wrong

Problem	Detail
Token never expires	Whatever is inside the kubeconfig — admin cert or SA token — is permanent
Base64 is not encryption	Anyone who can read GitHub Secrets decodes it instantly. It is plain text.

Problem	Detail
Scope is often too wide	Teams export admin kubeconfig to avoid RBAC debugging — full cluster access
No branch restriction	Any branch, any workflow in the repo can use KUBE_CONFIG_B64
Zero AWS audit trail	kubectl calls go directly to K8s API — CloudTrail has no record
Personal credential	Dev leaves team → their kubeconfig is still in production GitHub Secret

Leak Scenario

KUBE_CONFIG_B64 leaks → attacker decodes base64 → gets full kubectl access → permanent.
Remediation: find a new kubeconfig, re-encode, update secret. Meanwhile cluster is compromised.

Scenario 2 ServiceAccount Long-Lived Token

Create a dedicated SA → extract its JWT token → store as K8S_SA_TOKEN

How It Works

Instead of exporting your personal kubeconfig, you create a Kubernetes ServiceAccount specifically for GitHub Actions, bind it to a Role (or ClusterRole), extract its token, and store that token in GitHub Secrets. This feels more controlled — same underlying problem.

```
# Create SA and bind role
kubectl create serviceaccount github-deploy -n project
kubectl create rolebinding github-deploy-rb \
  --clusterrole=edit --serviceaccount=project:github-deploy -n project

# Extract token (Kubernetes < 1.24 auto-creates a secret)
SECRET=$(kubectl get sa github-deploy -n project -o jsonpath='{.secrets[0].name}')
TOKEN=$(kubectl get secret $SECRET -n project -o jsonpath='{.data.token}' | base64 -d)
SERVER=$(kubectl config view --raw -o jsonpath='{.clusters[0].cluster.server}')

# Store TOKEN, SERVER, CA_CERT as GitHub Secrets

# In the pipeline
kubectl config set-credentials github-deploy --token=$K8S_SA_TOKEN
kubectl config set-cluster my-cluster --server=$K8S_SERVER
kubectl apply -f k8s/ -n project
```

Is This Better Than Scenario 1?

Slightly. The scope is smaller (dedicated SA, not admin). But the mechanism is identical — you still stored a permanent credential in GitHub Secrets.

Problem	Detail
Token is still permanent	Kubernetes SA tokens (pre-1.24) have no TTL. They work forever.
Still stored in GitHub	K8S_SA_TOKEN is a permanent K8s credential sitting in GitHub Secrets
Scope creep happens	Teams start with 'edit'. Debugging breaks → add cluster-admin. Never revert.
Still no AWS trail	SA token calls go to K8s API server. AWS CloudTrail sees nothing.
Still no branch restrict	K8S_SA_TOKEN available to every branch and workflow
Remediation breaks CI/CD	Token leaked → delete SA to revoke → pipeline immediately stops working

Scenario 1 vs Scenario 2 — Are They Really Different?

Not fundamentally. Both store a long-lived Kubernetes credential in GitHub Secrets.
Scenario 1 = admin/personal kubeconfig → wider blast radius
Scenario 2 = dedicated SA token → smaller blast radius

Same mechanism. Same root problem. Different privilege scope.
The real problem — a permanent stored credential — is identical in both.

Scenario 3 ServiceAccount Short-Lived Token (TokenRequest API)

Generate a 1hr K8s token at runtime — but you still need AWS keys to bootstrap

How It Works

Kubernetes 1.20+ introduced the TokenRequest API. Instead of storing a permanent SA token, you generate a time-limited token (1hr) at the start of each pipeline run using 'kubectl create token'. The K8s token itself is short-lived. This sounds like the solution — until you ask the bootstrap question.

The Bootstrap Problem

To call 'kubectl create token' from GitHub Actions, you first need to authenticate to EKS. You cannot call kubectl against EKS without a valid kubeconfig. To get that kubeconfig, you must run:

```
aws eks update-kubeconfig --name my-cluster --region ap-south-1
```

And to run that AWS CLI command, you need AWS credentials. Which means you need to store:

What You Must Store in GitHub Secrets for Scenario 3

AWS_ACCESS_KEY_ID ← long-lived AWS credential
AWS_SECRET_ACCESS_KEY ← long-lived AWS credential

These are permanent. They never expire unless you manually rotate them.
You solved the Kubernetes token expiry problem.
You replaced it with a permanent AWS credential problem.

The full pipeline flow for Scenario 3:

```
# Step 1: Authenticate to AWS using stored long-lived keys
aws configure set aws_access_key_id $AWS_ACCESS_KEY_ID
aws configure set aws_secret_access_key $AWS_SECRET_ACCESS_KEY

# Step 2: Build kubeconfig using AWS creds
aws eks update-kubeconfig --name my-cluster --region ap-south-1

# Step 3: Generate short-lived K8s token (1hr)
TOKEN=$(kubectl create token github-deploy \
  --namespace project --duration=3600s)

# Step 4: Rebuild kubeconfig with the short-lived token
kubectl config set-credentials github-deploy --token=$TOKEN

# Step 5: Deploy
kubectl apply -f k8s/ -n project
```

You went through 5 steps and you still have AWS_ACCESS_KEY_ID in GitHub Secrets. The K8s token is short-lived — the bootstrap credential is not.

What Scenario 3 Gets Right

- The Kubernetes token expires in 1hr — if intercepted, limited window
- No permanent SA token stored in GitHub Secrets
- Slightly better audit than Scenarios 1 & 2 (AWS CloudTrail sees the update-kubeconfig call)

What Scenario 3 Still Gets Wrong

Remaining Problem	Why It Matters
AWS keys stored permanently	AWS_ACCESS_KEY_ID + AWS_SECRET_ACCESS_KEY in GitHub — these never expire
Worse blast radius than Scenarios 1&2	AWS keys can access your ENTIRE AWS account — not just EKS

Remaining Problem	Why It Matters
Still no branch restriction	Any branch can use the AWS keys to bootstrap and generate a token
Key rotation breaks pipelines	Rotate AWS keys → pipelines break → update 2 secrets under pressure
Extra complexity, same risk	5 steps instead of 1, but the stored credential problem is unchanged

The Bootstrap Problem — Visualised

Scenario 1: GitHub → [KUBE_CONFIG_B64] → kubectl → EKS
Scenario 2: GitHub → [K8S_SA_TOKEN] → kubectl → EKS
Scenario 3: GitHub → [AWS_ACCESS_KEY + SECRET_KEY] → aws eks update-kubeconfig
→ kubectl create token (1hr)
→ kubectl → EKS

All three have a [ ] — something permanent stored in GitHub.
You cannot escape the bootstrap problem without OIDC.

GitHub OIDC — How It Breaks the Bootstrap Problem

OIDC removes the [ ] entirely. GitHub proves its identity to AWS using a cryptographically signed JWT that it generates per run. There is nothing to store. Nothing to rotate. Nothing to leak.

```
GitHub Actions starts
    |
    |  GitHub generates a signed JWT (per run):
    |    iss: token.actions.githubusercontent.com
    |    sub: repo:jaiswaladi246/3-Tier-NodeJS:ref:refs/heads/main
    |    aud: sts.amazonaws.com
    |    exp: now + 5 min  ← one-time use
    |
    ▼  sent to AWS STS

IAM Trust Policy validates:
    ✓ JWT signature matches GitHub public key
    ✓ sub = repo:jaiswaladi246/3-Tier-NodeJS:ref:refs/heads/main
    ✓ aud = sts.amazonaws.com
    |
    ▼  issues temporary STS credentials [TTL: 15 min]

aws eks update-kubeconfig --name my-cluster --region ap-south-1
    |
    ▼  EKS Access Entry → AmazonEKSEditPolicy → namespace: project

kubectl apply -f k8s/   →   deployment succeeds
    |
    └── Credentials expire. Nothing to clean up.
```

GitHub Actions YAML:

```
- name: Configure AWS credentials (no keys needed)
  uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: ${{ secrets.AWS_ROLE_TO_ASSUME }}
    aws-region: ap-south-1

- name: Update kubeconfig
  run: aws eks update-kubeconfig --name my-cluster --region ap-south-1
```

AWS_ROLE_TO_ASSUME is a role ARN — a public identifier. It cannot authenticate anything on its own. It is not a secret.

All 3 Scenarios vs GitHub OIDC — Full Comparison

	Scenario 1 kubeconfig	Scenario 2 SA Long Token	Scenario 3 SA Short Token
Stored in GitHub?	Yes — kubeconfig	Yes — SA token	Yes — AWS keys
What's stored	Admin/SA token (K8s)	SA JWT token (K8s)	AWS_ACCESS_KEY_ID + AWS_SECRET_ACCESS_KEY
Credential expires?	Never	Never	Never (AWS keys don't)
K8s token expires?	Never	Never	Yes — 1hr
Bootstrap problem?	No — direct	No — direct	Yes — needs AWS keys first
AWS audit trail?	None	None	Yes — CloudTrail
Branch restriction?	None	None	None
Leak impact	Permanent cluster	Permanent cluster	Permanent AWS account
Compliance	Fails	Fails	Partial

The One-Line Summary

Scenario 1 — You store a permanent Kubernetes credential.
Scenario 2 — You store a permanent Kubernetes credential (smaller scope).
Scenario 3 — You solve K8s token expiry but store permanent AWS keys instead.
The bootstrap problem moves. It doesn't disappear.

GitHub OIDC — No stored credential. Identity proved at runtime. Bootstrap problem solved.